YEDİTEPE
ÜNİVERSİTESİ

CSE 224
Introduction to Digital Systems
Term Project Report

ASIC Implementation of a 5-Part Digital Design Using OpenLane and SkyWater Sky130A PDK

Ege Huriel
Assigned by Prof. Cem Ünsalan
June 15, 2026

Contents

1. Introduction

This report documents the CSE 224 Introduction to Digital Systems Term Project, which involves the design and ASIC implementation of five digital hardware modules using the OpenLane RTL-to-GDS flow and the SkyWater Sky130A, 130nm open-source Process Design Kit (PDK). Each part of the project builds upon the previous, beginning with a standalone Arithmetic Logic Unit (ALU) and culminating in a fully integrated half-CPU system with button debouncing and 7-segment display output.

All designs were originally developed for the Nexus 4 DDR FPGA board using Vivado and System-Verilog. For ASIC implementation, the source files were adapted to standard Verilog, removing FPGA-specific primitives, Xilinx attributes and initial blocks with are not supported in ASIC synthesis flows.

1.1 Tools and Technology

Tool / Technology	Version / Details
OpenLane	v1.0.2 (ff5509f)
Process Design Kit	SkyWater Sky130A
Standard Cell Library	sky130_fd_sc_hd
Linter	Verilator
Synthesis	Yosys
Place & Route	OpenROAD
Layout Viewer	KLayout 0.30.8
HDL	Verilog (IEEE 1364-2001)

1.2 Project Overview

Part	Top Module	Description	Clock
1	alu	8-bit ALU, 8	No
2	alu_flags	8-bit ALU + flag bits	No
3	register_file	16-bit register file	Yes
4	cpu_top	Half CPU (ALU + RegFile + Decoder)	Yes
5	full_system	Half CPU + Debouncer + 7-	Yes

2. Part 1 — 8-bit ALU

Part 1 implements a purely combinational 8-bit Arithmetic Logic Unit supporting eight operations selected by a 3-bit opcode. The design requires no clock and was synthesized as a combinational block in OpenLane with clock tree synthesis disabled.

2.1 Operations

op[2:0]	Operation	Description
0	ADD	A + B
1	SUB	A - B
10	MUL	A * B (full 16-bit result)
11	DIV	A / B (zero-division protected)
100	AND	A & B
101	OR	A | B
110	NOT	~A (unary, B ignored)
111	XOR	A ^ B

2.2 Port Description

Port	Width	Direction	Description
A	8-bit	Input	Operand A
B	8-bit	Input	Operand B
op	3-bit	Input	Operation select
Y	16-bit	Output	Result (16-bit for MUL)

2.3 Design Notes

The output is 16-bit to accommodate the full range of the multiplication result (8x8 = up to 65025, requiring 16 bits). For all other operations, the upper 8 bits are zero-padded. Division by zero is handled explicitly, returning 0x0000. The design uses a simple always @(*) block with a case statement, which maps cleanly to combinational logic during synthesis.

2.4 OpenLane Configuration

Since Part 1 is a purely combinational design, RUN_CTS was set to false and CLOCK_PORT was set to NULL. Both timing and design resizer steps were disabled to prevent the router from attempting timing driven optimizations on a clock-free design. The final die are was set to 300x300 um with a placement to density of 0.60.

3. Part 2 — 8-bit ALU with Flag Bits

Part 2 extends the ALU from Part 1 with four status flag output bits. These flags are essential for implementing conditional operations and control flow in a processor. The design remains fully combinational.

3.1 Flag Bits

Flag	Description	Condition
carry	Carry out	Set when ADD/SUB produces a carry out of bit 7
overflow	Signed overflow	Set when signed result exceeds 8-bit range
zero	Zero flag	Set when the 16-bit output is 0x0000
sign	Sign flag	Set when bit 7 of the result is 1 (negative)

3.2 Overflow Detection

Overflow detection is implemented using standard signed arithmetic rules. For addition, overflow occurs when both operands have the same sing but the result has a different sign. For subtraction, overflow occurs when operands have different signs and the result sign differs from the minuend. For multiplication, overflow is flagged when the upper 8-bits of the 16-bit product are non-zero.

3.3 OpenLane Configuration

The same combinational configuration as Part 1 was used with the die area increased to 400x400 um and placement density reduced to 0.40 to account for the additional flag logic. Design resizer step were also disabled to resolve routing congestion.

4. Part 3 — 16-bit Register File

Part 3 implements a general-purpose register file with 16 registers, each 16 bits wide. The design supports synchronous write and asynchronous read operations. No initial values are loaded, making it fully ASIC-compatible.

4.1 Port Description

Port	Width	Direction	Description
clk	1-bit	Input	Clock signal
rst	1-bit	Input	Synchronous reset, clears data_out
we	1-bit	Input	Write enable
address	16-bit	Input	Register address (lower 4 bits used)
data_in	16-bit	Input	Data to write into selected register
data_out	16-bit	Output	Data read from selected register

4.2 Implementation Notes

The register file is implemented as 16 individual flip-flop based registers rather than a memory array. This approach was chosen because Yosys synthesis large memory arrays into millions of gates, causing out-of-memory errors during the ABC technology mapping step. A flip-flop bank of 16 x 16-bit registers produces a manageable gate count while meeting the project requirements. The full 16-bit address port is preserved for interface compatibility, with the lower 4 bits used for indexing.

4.3 Timing Constraints

An SDC constraints file was provided specifying a 10ns clock period (100MHz), with 2ns input and output delays. Clock tree synthesis was enabled for this design. The GRT_ALLOW_CONGESTION flag w-as set to true to allow the router to proceed despite moderate congestion levels caused by the dense f-lip-flop structure.

5. Part 4 — Half CPU

Part 4 integrates the ALU, register file and an instruction decoder into a simple single-cycle CPU. The processor fetches instruction from an instruction memory register file, decodes them, reads operands from source registers, executes the ALU operation and writes the result back to a destination register.

5.1 Architecture

The half CPU uses four register file instances: one for instruction memory (read-only), two for reading source operands Ra and Rb and one for writing the result to the destination register Rd. Program Counter (PC) increments each cycle and halts when a HALT instruction (opcode 0xF) is encountered.

5.2 Instruction Format

Bits [15:12]	Bits [11:8]	Bits [7:4]	Bits [3:0]
opcode (4-bit)	Ra — Source A (4-bit)	Rb — Source B (4-bit)	Rd — Destination (4-bit)

5.3 ALU Operations

Opcode	Operation
0	ADD — Rd = Ra + Rb
1	SUB — Rd = Ra - Rb
10	SHL — Rd = Ra << 1
11	SHR — Rd = Ra >> 1
100	AND — Rd = Ra & Rb
101	OR — Rd = Ra | Rb
110	NOT — Rd = ~Ra
111	XOR — Rd = Ra ^ Rb
1111	HALT — PC stops incrementing

5.4 Submodule Hierarchy

Module	Instance	Function
register_file	instr_mem	Instruction memory (read-only)
decoder	dec	Splits 16-bit instruction into fields
register_file	reg_file_a	Reads operand Ra

Module	Instance	Function
register_file	reg_file_b	Reads operand Rb
alu	alu_inst	Executes arithmetic/logic operation
register_file	reg_file_d	Writes result to Rd

6. Part 5 — Full System

Part 5 integrates the halt CPU from Part 4 with a button denouncer, a one pulse generator and a multiplexed 7-segment display driver. The CPU result register is continuously displayed on the 7-segment display in hexadecimal format.

6.1 Submodule Description

Module	Instance	Description
cpu_top	u_cpu	Half CPU from Part 4
debounce	u_debounce	Synchronizes and debounces button input using a counter
one_pulse	u_one_pulse	Converts debounced signal to a single clock-cycle pulse
sevenseg_mux	u_sevenseg	2-digit multiplexed 7-segment display driver

6.2 Debouncer Operation

The debouncer module uses a two-stage synchronizer to eliminate metastability from the asynchronous button input, followed by a counter that must reach COUNT_MAX before the output is updated. This ensures that only stable, settled button states are passed to the downstream logic. The one-pulse module then converts the debounced level signal into a single-cycle pulse on the rising edge.

6.3 7-Segment Display

The sevenseg_mux module drives two 7-segment digits in a multiplexed fashion. A refresh counter cycles through the two digits, displaying the lower and upper nibbles of the 8-bit CPU result in hexadecimal. The anode select signal (an) is driven low for the active digit and the segment cathode signals (seg) are updated accordingly.

6.4 Port Description

Port	Width	Direction	Description
clk	1-bit	Input	System clock
rst	1-bit	Input	Active-high reset
btn_step	1-bit	Input	Raw button input
seg	7-bit	Output	7-segment cathode signals
dp	1-bit	Output	Decimal point (always off)
an	2-bit	Output	Digit anode select

7. OpenLane Implementation Flow

The OpenLane RTL-to-GDS flow was used to implement all five parts. The flow consists of linting, synthesis, floor-planning, placement, clock tree synthesis, routing and signoff steps. Each part required a config.json configuration file and a pin_order.cfg file for I/O placement.

7.1 Flow Steps

Step	Tool	Description
0 — Lint	Verilator	Syntax and semantic checking of HDL sources
1 — Synthesis	Yosys	RTL to gate-level netlist conversion
2 — STA	OpenSTA	Pre-layout static timing analysis
3 — Floorplan	OpenROAD	Die area and core area definition
4 — IO Placement	OpenROAD	Pin placement using pin_order.cfg
5 — Tap/Decap	OpenROAD	Well tap and decap cell insertion
6 — PDN	OpenROAD	Power distribution network generation
7-9 — Placement	OpenROAD	Global and detailed cell placement
10 — CTS	OpenROAD	Clock tree synthesis (sequential designs only)
11-13 — Routing	OpenROAD	Global and detailed routing
19-25 — Signoff	OpenROAD/Magic	SPEF extraction, STA, IR drop analysis
27-28 — GDS	Magic/KLayout	GDSII layout generation
33-36 — DRC/LVS	Magic/OpenROAD	Design rule and layout vs. schematic checks

7.2 Key Configuration Parameters

Parameter	Parts 1-2	Parts 3-5	Purpose
RUN_CTS	false	true (default)	Enable clock tree synthesis
CLOCK_PORT	null	"clk"	Identify clock port
DIE_AREA	300x300 um	500-600 um	Physical die dimensions
PL_TARGET_DENSITY	0.40-0.60	0.25	Cell placement density
GRT_ALLOW_CONGESTION	false	true	Allow routing congestion
QUIT_ON_SETUP_VIOLATIONS	false	false	Ignore timing violations
GLB_RESIZER_*	false	false	Disable timing resizers

7.3 FPGA-to-ASIC Adaptations:

The original lab source files were written in SystemVerilog targeting the Nexys 4 DDR FPGA board The following changes were made to ensure compatibility with the OpenLane ASIC flow:

FPGA (Original)	ASIC (OpenLane)	Reason
logic keyword	reg / wire	Yosys compatibility
always_ff / always_comb	always @(posedge clk) / always @(*)	Standard Verilog
initial blocks	Removed	Not synthesizable in ASIC
(* rom_style="block" *)	Removed	Xilinx-specific attribute
65536-entry memory array	16-register flip-flop bank	Avoids ABC OOM error
.xdc constraint files	config.json + pin_order.cfg	OpenLane format
switches / leds ports	Removed	FPGA board I/O only

8. Result

All five parts were successfully implemented through the full OpenLane RTL-to-GDS flow. Each design passed linting, synthesis, placement, routing, DRC and LVS checks. GDSII layout files were generated for all parts and verified with KLayout.

Part	Module	Die Area	DRC Violations	Flow Status
1	alu	300x300 um	0	PASS
2	alu_flags	400x400 um	0	PASS
3	register_file	500x500 um	0	PASS
4	cpu_top	500x500 um	0	PASS
5	full_system	600x600 um	0	PASS

8.1 Challenges and Solutions

Challenge	Solution
CLOCK_TREE_SYNTH deprecated in OpenLane v1.0.2	Replaced with RUN_CTS flag
DIODE_INSERTION_STRATEGY deprecated	Replaced with GRT_REPAIR_ANTENNAS
Routing congestion in Parts 1-2	Increased DIE_AREA, disabled design resizers
Segfault in timing resizer (Part 1)	Disabled GLB_RESIZER_TIMING_OPTIMIZATIONS
ABC out-of-memory for 65536-entry memory	Reduced to 16-register flip-flop bank
Setup violations on combinational designs	Set QUIT_ON_SETUP_VIOLATIONS to false
Port size mismatch warnings in Part 4	Added missing wire [15:0] instruction declaration

9. Conclusion

The project successfully demonstrated the full ASIC design flow from RTL to GDSII for five progressively complex digital hardware modules. Starting from a simple combinational ALU and building up to an integrated CPU system with peripheral I/O, all designs were implemented using the open-source OpenLane toolchain and the SkyWater Sky130A PDK.

The project highlighted key differences between FPGA and ASIC design flows, including the removal of FPGA-specific constructs, the handling of memory arrays in synthesis and the importance of proper timing constraints for sequential designs. All five parts passed DRC and LVS checks, producing clean GDSII layouts ready for fabrication.

9.1 Output Files

Part	GDS Output Path
1	designs/part1_alu/runs/.../results/final/gds/alu.gds
2	designs/part2_alu_flags/runs/.../results/final/gds/alu_flags.gds
3	designs/part3_regfile/runs/.../results/final/gds/register_file.gds
4	designs/part4_halfcpu/runs/.../results/final/gds/cpu_top.gds
5	5 designs/part5_full/runs/.../results/final/gds/full_system.gds